

任务图覆盖不到的级联损坏: 无冲突柔性作业车间中的时空预约闭包

QI YU, Southeast University, PR China

WEI LI*, Southeast University, PR China

当柔性作业车间的运输被建模为常数时间矩阵时, 车辆互不阻挡, 延误只是布局属性; 一旦 AGV 在显式走廊网络上以时间窗预约通行, 延误成为一车施加给另一车的量, 排程的可行性便同时写在任务依赖与预约表上。在同时持有任务依赖图与显式走廊预约表的这一支调度文献中, 影响域仍沿受影响工序重调度 (AOR) 的传统在任务依赖图上界定: 由故障工序或故障智能体作种子, 再按工序/指派邻域扩张。这一做法覆盖一类扰动 (机器故障、插单等), 却系统性地漏掉另一类——走廊阻断、路段降速等并不改动任何工序指派的扰动。对后者, 任务图上直接受扰集合为空, 框架会判「无需重调度」, 而预约表上已有大片占用失效, 并沿「谁挡了谁」向外级联。本文把这一缺口表述为: 任务图覆盖不到的级联损坏——不是检测漏检, 而是任务图这一表示本身不含预约级联通道。该缺口只在两张图分立的模型层级上存在: 铁路调度的 alternative graph 把闭塞区段即视为机器, 两图合一; 多智能体路径规划则没有任务图。

本文主张: 在无冲突路由设定下, 扰动的影响边界必须定义在时空预约图上。我们提出时空预约影响闭包 (STRC, Spatiotemporal Reservation Closure): 以与扰动时窗重叠的预约为种子, 沿等待阻塞、同车接续、同工件接续与机器链等依赖取传递闭包, 得到可释放、可冻结的损坏范围。让行边与 alternative arc 同源, 本文不主张它是新对象; 新的是它的用法——不选择弧, 而在既定排程之上取闭包, 使影响集从算法过程的副产品变为由图结构唯一确定的对象。在闭包上做有界修复 (改路, 失败则扩大释放域) 是把该对象用起来的手段; 「局部重规划 + 外侧冻结」与「失败扩域」两者在多智能体路径规划与局部重调度中均已标准做法, 本文不以为之创新。

实验取 5 算例 $\times$ 10 种子 (50 个算例-种子对), 在同一无冲突执行器与同挂钟协议下对照任务图边界臂与热启动全局重解臂。走廊阻断上任务图影响域在 50/50 上为空而闭包非空且原排程不可行; 同修复引擎下仅替换边界定义时, 任务图边界 0/50 恢复可行而闭包 50/50; 闭包的结构闭合性与闭包外侧字段的逐条不变均在 50/50 上成立; 修复耗时中位 2.8 ms, 较热启动全局重解低两至三个数量级, 改动的预约为 59%–66% 而后者为 97%–100%。不利读数如实报出: 完工时间上有界修复在全部 18 个预算点上劣于热启动全局重解, 不存在交叉点; 闭包亦不最小, 它覆盖决策时刻之后活预约的约 90%。本文因此不宣称在完工时间上优于全局搜索; 贡献在于指出并修正影响边界的划定对象, 并给出与显式预约表设定相匹配的局部恢复路径。

CCS Concepts: • **Theory of computation** → **Scheduling algorithms**; • **Applied computing** → *Industry and manufacturing*; • **Computing methodologies** → *Planning and scheduling*.

Additional Key Words and Phrases: 柔性作业车间调度, AGV 无冲突路由, 时空预约表, 影响闭包, 级联损坏, 局部重调度, 走廊阻断

ACM Reference Format:

Qi Yu and Wei Li. 2026. 任务图覆盖不到的级联损坏: 无冲突柔性作业车间中的时空预约闭包. 1, 1 (August 2026), 20 pages. <https://doi.org/XXXXXXX.XXXXXXX>

*corresponding author

Authors' Contact Information: Qi Yu, 230240012@seu.edu.cn, School of Cyber Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China; Wei Li, xchlw@seu.edu.cn, School of Computer Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

1 引言

1.1 背景与动机

考虑运输的柔性作业车间调度长期依赖一个便利假设: 工件在机器之间的移动时间是一张按位置索引的常数矩阵。该假设等价于默认车辆互不干扰——行驶时间是布局的固有属性, 而不是路由决策的结果。一旦有限车队共享稀疏走廊网络, 并以半开时间窗预约走廊资源, 上述默认便不再成立: 一辆车的让行会推迟另一辆车的到达, 进而推迟工序开工与后续预约。此时, 一份「可行排程」同时包含工序指派、排序与一张走廊预约表; 扰动不仅可以打坏任务, 也可以只打坏预约。

静态一体化排程的研究已经表明, 显式无冲突路由与柔性指派的耦合会改变指派优劣次序, 而把拥堵写进评价、在决策中查询冲突信息, 是构建高质量排程的关键路径 [9, 17, 19]。那些工作主要回答: 如何在信息完备时建出一份好的无冲突排程。车间运行中的问题不同: 在执行时刻 t_{now} 发生扰动之后, 损坏落在何处、改到何处才够——以及这件事能否在毫秒至百毫秒量级完成, 而不必每次都重新打开种群搜索。

1.2 任务图覆盖不到的缺口

在考虑运输的柔性作业车间动态调度这一支中, 影响域通常沿任务依赖图界定: 由直接失效的工序或智能体作种子, 再按工件后继、机器链或固定跳数 θ 扩张, 然后只对影响域内任务重优化。这一传统可上溯至受影响工序重调度 (AOR) [1], 是本文 R1 对照臂的原型 (第 2.1 节)。这一边界在「扰动本身就是任务事件」时往往合理——例如机器故障使若干工序无法按原指派执行。然而在显式预约模型里, 存在一类扰动, 其作用对象是走廊时窗而非工序指派。典型例子是走廊阻断: 某走廊在 $[t_a, t_b)$ 不可通行。阻断不修改任何机器指派或工序顺序, 因此任务图上的直接受扰集合 $T_{\text{direct}} = \emptyset$; 若框架以「空影响域」推出「无需重调度」, 则会系统性漏报——原排程中落在该时窗的预约已经失效, 并可能通过等待阻塞关系迫使后续车辆改道或推迟, 形成预约表上的级联。

本文强调用语: 所谓「覆盖不到」, 指的是任务图这一表示天然不含预约级联通道, 而不是传感器漏检或算法偶然漏报。在常数运输矩阵模型里, 这类损坏甚至无法被写下——走廊不是资源, 预约表不存在。因此, 问题本身附着于无冲突路由与显式预约设定; 这不是应用上的窄化, 而是表述该缺口所必需的模型前提。

据此, 我们区分两类扰动 (后文形式化):

- **A 类** (触碰任务图): 机械臂故障、加工时间显著偏差、插单, 以及车辆故障——任务图种子非空; 此时任务图边界与预约闭包都能看见该扰动, 不是本文主检验对象。车辆故障归入 A 类的理由见第 3.3 节: 它虽不改工序指派, 却使该车承运的工序不可执行, 经车-任务映射在任务图上留下非空种子。
- **B 类** (只触碰预约表): 走廊阻断或降速、局部通行权临时取消等——任务图种子为空, 预约表种子与闭包非空。

B 类构成问题层的一击反例: 不是「需要更好的修复启发式」, 而是「在两张图并存的这一支模型里, 影响边界一直被划在看不见该扰动的那张图上」。这个错位需要一个相当特殊的前提——模型必须同时持有任务图与让行图, 却只用其中一张划界。缺少任一半都不会出现: 铁路调度里闭塞区段就是机器, 两图合一, 区段阻断天然属于 A 类; 多智能体路径规划里任务外生, 没有任务图可供划错。故本文的主张写作「这一支」, 而非对局部重调度文献的全称否定 (第 2.5 节)。图 1 对照这一缺口: 左侧任务图种子为空因而判「无需重调度」; 右侧预约表上阻断窗命中种子并沿让行/接续边形成闭包。

1.3 主张与方法预告

本文主张三句话。第一, 在无冲突柔性作业车间中, 扰动影响边界应定义在时空预约图上, 而非仅定义在任务依赖图上。第二, 正确的影响对象是时空预约影响闭包 (STRC): 以失效预约为种子, 沿预约阻塞与调度接续关系取传递闭包; 闭包之外的预约在结构上与种子无依赖, 从而「局部」对应一个可检验的对象, 而不是启发式剪枝半

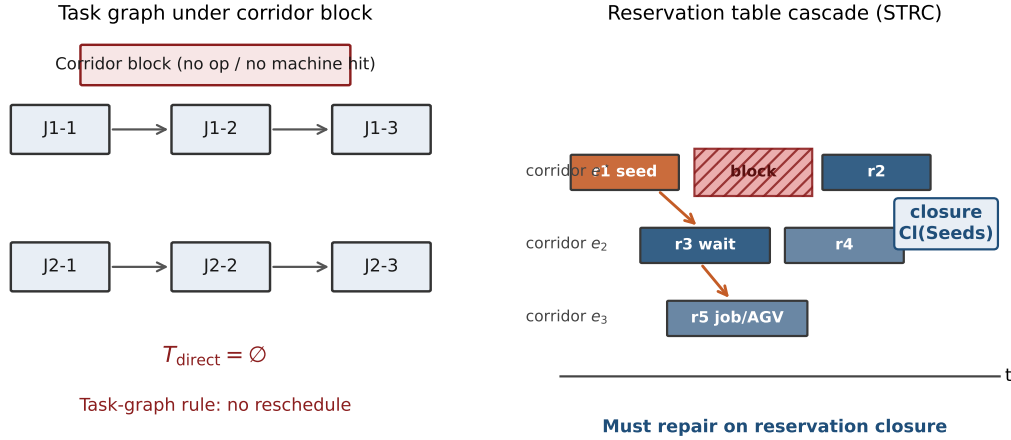


Fig. 1. 动机示意。左: 走廊阻断不命中任何工序/机器, 任务图直接集为空。右: 同一扰动在预约表上产生种子, 并沿阻塞与接续关系级联为时空预约闭包。

径。第三, 闭包上的有界修复——释放闭包内预约、外侧冻结、原车改路, 失败则沿工件/车辆链扩大释放——是闭包的应用; 升级阶梯中更贵的协调 (换车、改派、改序) 可以随后展开, 但不是本文创新的挂靠点。

三句话共有一个立场, 值得单独点明: **影响闭包是一次测量, 不是一个决策**。本文不用阻塞关系去制导搜索——那样做买的是「改进一个决策」, 其收益要经受关键链的稀释 (消去一处走廊等待, 往往只是把另一处约束顶成紧约束)。本文用同一份阻塞关系去划定损坏范围——买的是「改进一次测量」, 而测量不经受稀释: 闭包要么包含了全部必须释放的预约, 要么没有, 这与关键链上顶上来的是谁无关。第 2.2 节用同一句话区分本文与铁路 alternative graph (那里选弧本身就是决策变量); 第 1.5 节用同一句话说明本文与静态一体化工作对阻塞信息的不同用法。

与「每次扰动都热启动全局重解」相比, STRC 路径回答的是另一侧问题: 在紧响应预算下能否快速恢复可行, 并在结构上把修改限制在闭包内。外侧预约字段的逐条不变是给定边集与预提交成功时的充分条件 (命题 3), 不是无条件的恒等式; 相对全局重解, 闭包路径改动更少、响应更快。实验将表明二者的分工在响应时间一侧而不在解质量一侧——闭包修复以极短耗时恢复可行; 全局重解在本文测到的每一个挂钟预算点上都取得更优完工时间——因而正确叙事不是「取代搜索」, 而是「测量损坏范围, 再决定用多便宜的协调就够」。

1.4 贡献

按问题层、框架层、机制层、方法层组织, 本文贡献如下。

- (1) **问题层: 指认任务图覆盖不到的级联损坏**。在显式走廊预约下形式化 B 类扰动: 按任务图直接集的常规定义, 走廊阻断给出 $T_{\text{direct}} = \emptyset$; 同时证明当预约种子非空时原排程仍不可行, 从而「空影响域 $\Rightarrow$ 无需重调度」失效。该缺口既不在常数运输时间模型中可表述, 也不在两图合一的模型 (如铁路) 中出现; 它需要「两张图分立、却只用一张划界」这一特定层级。
- (2) **框架层: 先测量、再有界协调**。将动态恢复组织为「闭包划定释放集 $\rightarrow$ 外侧冻结 $\rightarrow$ 闭包内最低必要协调」, 与「打开种群搜索做全局重解」对照而不混为一谈。测量优先于协调; 修复是框架的第二步而非标题主语。

- (3) **机制层: 时空预约影响闭包 (STRC)**。定义种子、依赖边与传递闭包; 证明结构闭合性 (闭包外不是闭包内顶点的出边后继), 并给出外侧字段不变的充分条件 (预提交成功且不扩域时)。此处新的不是让行关系这一对象 (见第 2.2 节), 而是把影响集由算法过程的副产品改造成由图结构唯一确定的对象: 它可陈述闭合性、可测量规模、可作为不同修复引擎的公共输入。同引擎消融表明贡献来自边界定义而非某次改路启发式的细节。
- (4) **方法层: 闭包上的有界恢复与对照协议**。实现第 1 级改路及失败扩域再修 (两者的机制形态均非本文首创, 见第 2.1、2.3 节); 在统一无冲突执行器与同挂钟协议下, 对照任务图边界臂与热启动全局重解臂, 报告可行性、耗时、预约变动与完工时间, 并给出扰动规模 φ 扫描下的互补关系。
- (5) **一张填满的覆盖矩阵, 以及一条不利读数**。在 50 个算例-种子对上, 把四类扰动 (走廊阻断、路段降速、车辆故障、机械臂故障) 与两种边界定义交叉, 给出任务图边界何时为空、闭包相对任务图释放集何时严格更大的完整读数。同时如实报告: 完工时间上有界修复对热启动全局重解全区间劣势、无交叉点; 闭包不最小, 覆盖 t_{now} 之后活预约的约 90%。

1.5 与静态一体化排程工作的关系

本文可独立阅读。若读者熟悉同一场景下的静态闭环双层排程工作, 则二者关系可一句话概括: **同一车间、同一预约表; 前者用预约表构建排程, 本文用预约表定位损坏**。共享对象是显式路网、走廊独占与半开时窗预约; 区别在时机与算法形态——静态工作在 $t = 0$ 做种群搜索以最小化完工时间, 本文在 t_{now} 做单解有界恢复以恢复可行并把修改锚定在预约闭包上。

有一处表面上的不一致需要点破。那类静态工作中, 用阻塞关系去制导搜索 (以「谁挡了谁」的凭证引导邻域算子) 往往收效甚微甚至为负; 而本文复用的正是同一套阻塞关系。二者并不矛盾, 因为买的东西不同: 制导搜索买的是「改进一个决策」, 收益要经受关键链的稀释——消去一处走廊等待, 常常只是让另一条约束项上来成为紧约束, 于是净改进被吃掉; 本文买的是「改进一次测量」, 而测量不经受稀释——闭包要么含有全部必须释放的预约, 要么没有, 这与关键链上项上来的是谁无关。**影响闭包是一次测量, 不是一个决策**; 同一份阻塞信息在这两种用法下有不同的期望回报, 恰是这条判据的两侧证据。独立阅读时, 上述静态工作视为相关文献即可。

1.6 篇章结构

第 2 节按「影响边界划在什么对象上」分四层回顾文献, 并划清本文不主张什么。第 3 节给出系统模型、扰动二分与恢复问题陈述。第 4 节定义 STRC 并陈述包含性 (图 2)。第 5 节描述闭包上的有界修复与扩域策略 (图 3)。第 6 节报告实验, 并以案例甘特衔接统计与时间轴。第 7 节总结, 并分节给出局限与后续工作。

2 相关工作

影响边界的划定并非新问题; 真正区分各支文献的是边界划在什么对象上。据此本节分四层: ① 任务图或时间轴 (2.1); ② 让行图, 但与任务图合为一张 (2.2); ③ 只有让行图而无任务图 (2.3); ④ 两张图并存, 却仍只用任务图划界 (2.4)——最后一层是本文的位置。表 1 汇总。

2.1 任务图与时间轴上的影响边界

在任务依赖图上划界的原型是**受影响工序重调度** (Affected Operations Rescheduling, AOR)。Li 等人 [10] 以调度二叉树与 MRP 的净改变概念只修订「需要修订的工序」; Abumaizar 与 Svestka [1] 给出完整的 AOR 算法——只重排受扰动直接或间接影响的工序, 其余保持原次序, 并以效率与稳定性两个维度同完全重调度、右移重调度对比; Subramaniam 与 Raheja [16] 的 mAOR 把它从单一机器故障推广到插单、工时偏差、订单取消等多种扰动。本

文的 **R1 对照臂就是 AOR**: 以失效工序为种子, 沿工件后继与机器链扩张, 再只重排影响域内的部分。把它写成对照不是因为它弱, 而是因为它这是这一支的标准做法, 且其种子定义直接决定了本文要检验的那件事。

沿时间轴划界是同样成熟的另一条路。**match-up** 调度 [3] 在扰动点之后选一个匹配时刻, 只重排两者之间的部分, 并给出该策略最优的条件; Aktürk 与 Görgülü [2] 用反馈机制确定匹配点。**Local Rescheduling** [8] 则把时间窗增量式扩张直到找到可行解, 并明确指出 AOR 与 **match-up** 都绑定在生产特有的问题结构上。右移与反应式规则响应快但不给出显式边界 [4]; 滚动时域与分解式方法在规模与质量之间折中 [22, 24]; 两篇综述系统梳理了完全重调度与修复式重调度的取舍 [14, 20]。

这一层给出两点纪律。其一,「应当划一个边界」不是本文的主张, 它至少有三十年历史; 本文能主张的只是边界该划在哪个图上。其二,「失败后扩大释放域」不是本文的机制创新——LRS 已是沿时间窗扩张, 本文第 5.2 节沿依赖边扩张只是同一思路换了扩张方向。这一层的共同隐含前提是: 扰动首先是一个任务事件。一旦扰动只使走廊时窗失效而不改指派, 该前提不再成立, 任务图种子可以为空, 而排程已不可行。

2.2 让行图已有先例: alternative graph 与铁路实时重调度

「谁在排他资源上先于谁」这一关系被显式建成图, 在铁路调度中已有二十余年历史。Mascis 与 Pacciarelli [12] 的 **alternative graph** 把析取图推广为: 顶点是作业在闭塞区段上的占用, 固定弧表达路径先后, 成对的 **alternative arc** 表达「谁先通过该区段」。D'Ariano 等人 [5] 以该模型做实时列车重调度, 在扰动后重算无冲突时刻表并最小化与原时刻表的偏离。本文的让行边与 **alternative arc** 是同一个对象, 必须先承认这一点。两处区别决定了本文仍然成立:

- **用途不同: 选弧 vs 取闭包**。在 **alternative graph** 里, 弧的选择本身是决策变量——求解即选一组弧使图无正环且目标最优。本文不选择弧: 让行边由已经落盘的那份排程唯一确定, 本文在这一既定选择之上取传递闭包, 回答的是另一个问题——扰动之后哪些承诺必须作废。用一句话来说, **影响闭包是一次测量, 不是一个决策**。
- **模型层级不同, 而这个差别正是本文问题的来源**。在铁路模型里闭塞区段就是机器, 任务图与让行图合为一张; 因此「区段阻断」在那里天然属于 A 类, 不存在任务图看不见的问题。本文的缺口只在两张图分立的模型里存在。这既是对该文献的礼让, 也是本文问题得以成立的前提。

2.3 无任务图的局部重规划: MAPF 的 Replan Affected

在多智能体路径规划中,「只为受影响者重规划、其余视作动态障碍」已是标准做法。Švancara 等人 [18] 在在线 MAPF 中明确给出 **Replan All / Replan Single / Replan Affected** 三档, 并以在线独立性检测确定受影响集合; Malka 等人 [11] 在地图不准确的设定下同样「只为受观测变化影响的智能体重规划」。无冲突路径规划把时空占用作为一等约束, 但任务集合与端点通常由外部给定 [23]。

因此「闭包内重规划 + 闭包外冻结」这一机制形态不是本文的创新, 本文不作此声称。MAPF 之所以不会遇到本文的问题, 是因为它根本没有任务图: 任务外生, 不存在「任务图看不见」这回事。它与本文的区别也不在机制, 而在影响集的性质——独立性检测的输出是算法过程的副产品, 随实现与终止条件而变; 闭包是由图结构唯一确定的对象, 可陈述闭合性、可测量规模、可作为不同修复引擎的公共输入 (第 4.2 节)。

2.4 两张图并存场合: 考虑运输的柔性作业车间

考虑运输车辆的柔性作业车间调度已有系统综述 [21]。近期组合优化工作仍普遍把行驶时间取为按位置索引的常数矩阵, 并在此之上发展约束规划、元启发式与知识引导搜索 [6, 13, 15]。在该建模约定下, 车辆互不占用同一时空资源, 延误不是一车施加给另一车的量; 走廊阻断这类事件甚至没有对应的模型对象。因此,「任务图覆盖

Table 1. 按「影响边界划在什么对象上」定位。最后一列是本文与该层的关系; 前三层本文均不主张新颖性。

层	代表工作	边界划在	有任务图?	与本文的关系
① 任务图 / 时间轴	AOR [1, 10, 16]; match-up [2, 3]; LRS [8]	工序集合 / 时间窗	有	R1 即 AOR 。边界划定与失败扩域皆非本文首创
② 让行图, 两图合一	alternative graph [12]; 铁路实时重调度 [5]	闭塞区段占用	有, 但与让行图同一张	让行边同源。区别: 选弧 (决策) 与取闭包 (测量); 且区段即机器, 故无本文缺口
③ 让行图, 无任务图	Online MAPF 的 Re-plan Affected [11, 18]; LMAPF [23]	受影响智能体	无 (任务外生)	「局部重规划 + 外侧冻结」为其标准做法, 本文不作机制创新; 区别在影响集是过程还是对象
④ 两图并存, 只用任务图	常数矩阵 FJSP-AGV [6, 13, 15, 21]; 一体化排程 [9, 17, 19]	工序集合	有, 且另有预约表	本文的位置 。边界与损坏的载体错位

不到的预约级联」在常数矩阵文献中既不会被提出, 也无法被检验——这不是那些工作的缺陷, 而是本文问题附着于另一建模层级的原因。

一旦下层改用显式走廊预约, 两张图就同时存在了。制造场景中的拥堵感知派车以局部密度等代理量驱动选车, 而不回头改机器指派 [7]。把柔性指派与无冲突运输真正耦合的工作相对较少: van Os 与 Basten [19] 形式化了带无冲突运输约束的柔性作业车间调度问题, 并以分解方法给出可证最优基准; 两阶段方案先固定排产再消解车辆冲突 [17]; 亦有以学习层排产、路径层消冲突并以有限反馈连接的框架 [9]。这些工作主要回答静态或准静态下如何构建可行且高质量的一体化排程, 而把执行期扰动留给了 2.1 那一套任务图边界。这正是本文缺口所在的那一格: 模型同时持有工序/机器链与显式走廊预约, 却沿用只在任务图上划界的传统。

2.5 本文定位, 以及本文不主张什么

综上, 本文的主张必须收窄到第 ④ 层: 在同时持有任务依赖图与显式走廊预约表的这一支调度文献中, 影响域仍沿任务图界定, 而这对 B 类扰动系统性失效。写成对「局部重调度文献」的全称否定是不成立的——第 ② 层不会犯这个错 (两图合一), 第 ③ 层不可能犯 (没有任务图), 缺口只在第 ④ 层这个特定的模型层级上存在。

相应地, 本文不主张以下三件事, 后文亦不再以创新的口吻提及: (i) 「只重规划受影响者、其余冻结为障碍」这一机制形态 (第 ③ 层已有); (ii) 「单次修复失败后扩大释放域」(LRS 已有, 只是沿时间窗而非沿依赖边); (iii) 「用让行关系组织冲突」这一建模对象 (alternative graph 已有)。本文主张的是: 该边界应改画在预约图上, 并且这一改画可以被定义成一个有闭合性、可测量的对象, 而不是又一条启发式规则。

3 问题描述

3.1 系统设定与记号

我们采用带无冲突运输约束的柔性作业车间设定 [19] 的执行期版本, 记号与静态闭环解码器对齐, 此处只保留后文需要的对象。

车间路由图 $G = (V, E)$ 强连通; 每条走廊 $e \in E$ 为排他资源, 通行时间 τ_e 已知。占用记为半开区间。 n 个工件、机器集合 M 、车队 $K = \{1, \dots, N_A\}$ 。工序 O_{ji} 的候选机器集为 $\Omega_{ji} \subseteq M$ 。一份完整排程

$$S = (x, \pi, y, R)$$

Table 2. 本文常用记号

符号	含义
$G = (V, E)$	走廊路网
R	走廊预约表 (半开占用区间集合)
G_T / G_R	任务依赖图 / 预约依赖图
S^0, S'	扰动前排程、恢复后排程
$\mathcal{D}, t_{\text{now}}$	扰动事件、发生 (决策) 时刻
$T_{\text{direct}}, T_{\text{impact}}$	任务图直接/ θ -跳影响域
$\text{Seeds}(\mathcal{D})$	与扰动重叠的预约种子
$\text{Cl}(\cdot)$	时空预约影响闭包 (STRC)
φ	受扰未来工序比例 (规模扫描)

包含机器指派 x 、工序扫描序 π 、车辆指派 y , 以及预约表

$$R = \{r = (e, [t^s, t^e), k, u)\},$$

其中 $k \in K$ 为车辆, u 为任务标签 (空载/满载段)。路径只占用 R 报告为空闲的时窗, 从而构造性保持无冲突。

任务依赖图 $G_T = (T_T, E_T)$ 以工序 (或工序级任务 id) 为顶点, 边为工件前后约束; 它不包含走廊占用之间的阻塞边。预约依赖图 $G_R = (R, E_R)$ 的顶点为 R 中的预约记录, 边集见第 4 节。

3.2 假设: 从静态完备到执行期扰动

静态构建阶段可沿用「信息完备、构造性可行、计划期内无故障」等约定。本文打开执行期扰动, 其余物理假设保持不变以便归因:

- A1. 初始排程可行。 S^0 在扰动前通过无冲突校验; 预约表与工序时间表一致。
- A2. 扰动在 t_{now} 被观测。已完成占用不回溯修改; 只重协调 $t_{\text{end}} > t_{\text{now}}$ 的未来预约与未完工工序。
- A3. 走廊仍为排他半开资源。阻断以附加占用 (或禁行窗) 注入路由层。
- A4. 缓冲与同构车队等与静态设定一致。有限缓冲、充电等仍不建模。
- A5. 允许的恢复动作。本文实现级默认: 保持原机器指派与扫描序, 允许改路; 失败时扩大释放集。换车/改派/改序留作升级阶梯的后续级, 实验中全局重解对照臂可改染色体。

3.3 扰动二分

定义 1 (A/B 类扰动). 若扰动直接使某些工序的指派或可执行性失效 (机械臂故障、显著工时偏差、插单、车辆故障等), 称其为 **A 类**; 若扰动只使某些走廊时窗不可用、而不使任何工序不可执行 (走廊阻断、路段降速等), 称其为 **B 类**。

注 1 (车辆故障为何属于 A 类). 车辆故障不改动任何工序的机器指派, 乍看应与走廊阻断同类。但定义 1 的判据是可执行性而非指派: 一辆车抛锚后, 它承运的那些工序在原计划下无法开工, 故经「车辆 $\rightarrow$ 所承运工序」的映射, 任务图上留有非空种子。这一归类是保守的, 它把一类边缘情形判给了对本文不利的一侧——A 类是任务图边界能看见的那一类。需要指出, 只有当重调度框架维护了车-任务映射时该种子才非空; 若框架只跟踪工序与机器 (AOR 的常见实现即如此), 车辆故障同样会漏报。本文不依赖这一点立论, 第 6.5 节按维护了该映射的强口径报告 A 类读数。

定义 2 (任务图直接集与影响域). 对扰动 $\mathcal{D}$, 令 $T_{\text{direct}}(\mathcal{D})$ 为被故障智能体或失效工序直接命中的任务 id 集合。对 B 类走廊扰动约定 $T_{\text{direct}} = \emptyset$ 。在 G_T 上从 T_{direct} 出发做深度至多 θ 的扩张, 记为 $T_{\text{impact}}(\mathcal{D}, \theta)$ 。

定义 3 (预约种子). 种子集 $\text{Seeds}(\mathcal{D}) \subseteq R$ 按扰动类型给出, 一律只取 $r.t^e > t_{\text{now}}$ 的预约。

- (a) 走廊阻断/降速 $\mathcal{D} = (e, [t_a, t_b]): \{r : r.e = e, [r.t^s, r.t^e] \cap [t_a, t_b] \neq \emptyset\}$. 多微阻断时取各窗种子之并。两者共用这条规则 (降速在当前实现中按整段不可用处理), 故它们给出同一个种子集——第 6.5 节据此说明为何不把二者算作两次独立验证。
- (b) 车辆故障 $\mathcal{D} = (k)$: 该车在 t_{now} 之后的全部预约 $\{r : r.k = k\}$ 。
- (c) 机械臂故障 $\mathcal{D} = (m, F)$, F 为该机上尚未完工的工序: 对每个工件取 F 中最小工序号 $i_{\min}(j)$, 种子为各工件自该工序起 (含) 的全部未来预约。之所以沿工件链向后取而不只取该工序自身, 是因为进站运输可能已在 t_{now} 前结束, 若只取自身, 闭包会短于任务图影响域, 对照即不公平。

命题 1 (B 类上的任务图空洞). 设 $\mathcal{D}$ 为走廊阻断。若采用定义 2 对 B 类的约定, 则 $T_{\text{direct}}(\mathcal{D}) = \emptyset$, 从而对任意有限 $\theta \geq 0$ 有 $T_{\text{impact}}(\mathcal{D}, \theta) = \emptyset$ 。若此外 $\text{Seeds}(\mathcal{D}) \neq \emptyset$, 则 $\mathcal{S}^0$ 在 $\mathcal{D}$ 下不可行。

PROOF. 由定义 2, 走廊阻断不命中任何故障智能体或失效工序, 故 $T_{\text{direct}} = \emptyset$ 。 θ -跳扩张的起点为空, 故 $T_{\text{impact}} = \emptyset$ 。若存在 $r \in \text{Seeds}(\mathcal{D})$, 则 r 的占用区间与禁行窗相交。将 $\mathcal{D}$ 解释为该走廊上的强制占用后, r 与禁行窗在同一排他资源上时间重叠, 违背半开时窗无冲突条件, 故 $\mathcal{S}^0$ 不可行。□

命题 1 即问题层的一击反例。前半句是任务图直接集对 B 类的约定推论 (指出任务图表示覆盖不到走廊事件); 非平凡的是后半句——种子非空时排程已不可行, 因而「空影响域 $\Rightarrow$ 无需重调度」在无冲突设定下不成立。该反例不依赖任何修复算法; 实验第 6 节给出算例规模。

3.4 恢复问题

问题 1 (预约表感知的有界恢复). 给定可行排程 $\mathcal{S}^0$ 、扰动 $\mathcal{D}$ 与时刻 t_{now} , 求 $\mathcal{S}'$, 使得: (i) $\mathcal{S}'$ 在 $\mathcal{D}$ 下通过无冲突校验; (ii) 未释放预约的字段尽可能与 $\mathcal{S}^0$ 一致; (iii) 墙钟响应时间受预算约束。次级目标为完工时间 $C_{\max}(\mathcal{S}')$ 。

框架层将求解拆为两步: 测量——计算释放集 $U \subseteq R$; 协调——仅对 U 内运输重规划, 外侧冻结。本文测量用 $\text{Cl}(\text{Seeds})$; 对照测量用任务图影响域诱导的预约子集。

4 时空预约影响闭包

4.1 预约依赖边

定义 4 (预约依赖图). 顶点为 R 中预约。有向边 $r \rightarrow r'$ 表示「 r 失效可能迫使 r' 重规划」, 包括:

- (1) 让行边: 同走廊、异车、时间重叠, 且先进入者指向后进入者;
- (2) 同车后继: 同一车辆按时间序相邻的两条预约;
- (3) 同工件后继: 同一工件工序 i 的预约指向工序 $i+1$ 的预约;
- (4) 同机后继: 同一机器时间轴上相邻两道工序所诱导的预约之间的边。

由此得到 $G_R = (R, E_R)$ 。

定义 5 (STRC). 对种子集 $S \subseteq R$ 、视界 H 与时刻 t_{now} , 记活预约为

$$R^\circ = \{r \in R : r.t^e > t_{\text{now}}, r.t^s < H\},$$

并令 G_R° 为 G_R 在 R° 上的诱导子图。时空预约影响闭包定义为

$$\text{Cl}(S) = \{v \in R^\circ : \exists s \in S \cap R^\circ \text{ 使 } s \rightarrow^* v \text{ 于 } G_R^\circ\},$$

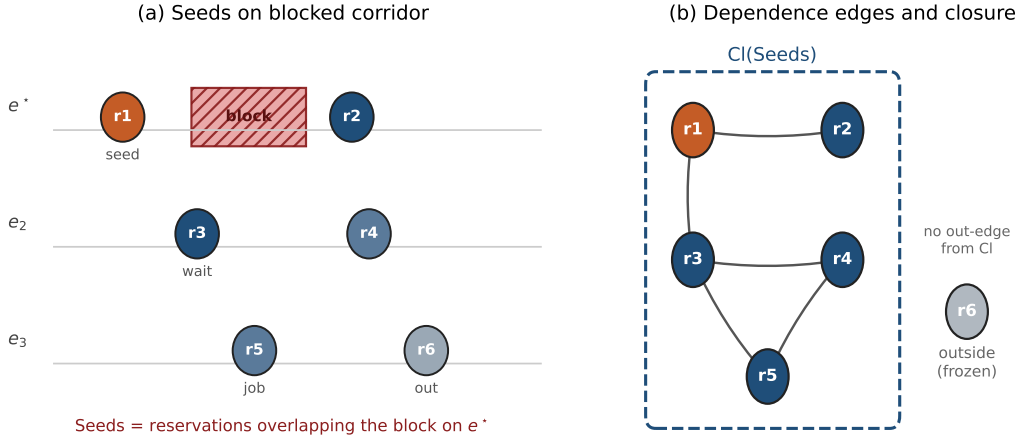


Fig. 2. 时空预约闭包构造示意。(a) 走廊阻断命中种子预约; (b) 沿让行/接续边取传递闭包 $Cl(Seeds)$, 外侧预约定向冻结。

即从活种子出发沿出边可达的全部活顶点(含种子自身)。若以邻接表表示 G_R° , 则可用队列在 $O(|R^\circ| + |E_R^\circ|)$ 时间内算出。

图 2 给出玩具示意:(a) 阻断窗与种子;(b) 依赖边与闭包外壳, 闭包外顶点不是闭包内顶点的出边后继(命题 2)。

对照的任务图边界臂(记 $R1$) 将 T_{impact} 映射为预约释放集: 凡任务标签属于影响域且 $t^e > t_{\text{now}}$ 的预约。由命题 1,B 类上该集合为空; 闭包臂(记 $R2$) 取 $U = Cl(Seeds)$ 。

4.2 包含性

命题 2 (结构闭包性). 对任意 $S \subseteq R$, 在 G_R° 中不存在边 $u \rightarrow v$ 满足 $u \in Cl(S)$ 且 $v \in R^\circ \setminus Cl(S)$ 。因而, 从 $Cl(S)$ 出发沿出边无法到达闭包外的活预约。

PROOF. 若存在这样的边 $u \rightarrow v$, 则由 $u \in Cl(S)$ 知存在活种子 s 使 $s \rightarrow^* u$ 。拼接 $u \rightarrow v$ 得 $s \rightarrow^* v$, 故按定义 5 应有 $v \in Cl(S)$, 矛盾。□

命题 2 说明「局部」对应一个图论闭包对象: 闭包外顶点不是任何闭包内顶点的直接后继。它不声称 G_R 已枚举物理世界中一切可能耦合——边集是建模选择; 一旦边集固定, 闭包性就是定义的推论, 而不是启发式半径。

注 2 (本文不作最小性主张). 闭包性说的是闭包足够大, 没有说它不过大。二者是两个方向, 后者本文不主张, 理由有三, 按分量排列。

其一, 最小释放集的定义依赖于修复引擎, 而闭包刻意不依赖。称释放集 U 为最小, 须指明「在何种修复能力下 U 足以恢复可行」。允许改派与改序时的最小 U , 与只允许改路时的最小 U 不是同一个集合。闭包由 G_R 唯一确定, 与引擎无关——这正是它能作为不同引擎公共输入的原因, 也是它无法同时具备最小性的原因。

其二, 四类边是充分方向的过近似。让行边按「同走廊、异车、时间重叠」建立, 只要两条预约在时间上重叠即连边, 而不判断后者是否真的只能靠等待前者通过——若走廊网络在该处另有旁路, 后者改道即可, 本不必进入释放集。同理, 同车后继边把一辆车在 t_{now} 之后的整条链连通, 而一次早期延误未必传得到链尾。这两处都会使闭包严格大于必要。

其三, 实测证实闭包确实不小。第 6.5 节报告, 走廊阻断下闭包平均覆盖 t_{now} 之后活预约的约 90%; 换言之, 相对「释放全部未来预约」这一平凡上界, 闭包省下的不足一成。有界修复真正省下的是不动过去与不重开搜索, 而不是把未来切得很小。这一点在第 6.8 节与局限一节中再次点明, 不应被「局部」「有界」这两个词误导。

因此本文的定位是: 闭包给出一个由结构唯一确定、可复现、足以恢复可行的释放集, 并附带闭合性保证; 逼近最小释放集是另一个问题, 列为后续工作 (第 7.3 节)。

命题 3 (外侧字段不变的充分条件). 设释放集 $U = \text{CI}(S)$, 并设第 1 级协调按第 5.1 节执行: (i) 禁行窗已写入路由层; (ii) 对每个 $r \in R^0 \setminus U$, 将其原字段 $(e, [t^s, t^e], k)$ 预提交到新预约表且预提交成功; (iii) 仅对 U 内运输调用路由器生成新占用; (iv) 最终排程通过无冲突校验。则在恢复后的预约表 R' 中, 每个外侧任务标签对应的占用字段与 S^0 一致。

PROOF. 由 (ii), 外侧预约在重放开始前已按其原区间写入新表, 且未与禁行窗冲突 (否则预提交失败)。步骤 (iii) 只为 U 内任务提交新占用, 不删除或改写已提交的外侧区间。因此, 对外侧任务标签, 新表中的 $(e, [t^s, t^e], k)$ 与预提交值相同, 亦即与 S^0 相同。校验通过仅排除新占用与既有占用冲突, 不改变已提交字段。□

注 3 (前提 (ii) 与 (iii) 各自会怎样失效). 若预提交因「外侧与禁行窗重叠」失败, 则 S 未能覆盖全部直接命中预约, 闭包种子不完备。若协调因运输-工序衔接失败而触发第 5.2 节扩域, 则新的释放集 $U' \supsetneq U$, 命题 3 只对最终未释放的外侧成立。扩域以可行性优先于最小扰动, 实验中单独报告。还有一种失效不在命题的可控范围内, 但在实现上极易发生: 前提 (iii) 要求路由器只为 U 内的运输生成新占用。若实现以比预约更粗的单位 (例如整道运输、整个工序) 决定是否重规划, 则 U 外的占用会被顺带改写, 命题的结论随之落空——不是命题错了, 而是前提没被满足。注 4 记录了本文在这一点上踩过的坑。

4.3 结构可预测性

闭包规模不是可调参数, 而是由排程与网络结构决定的量: 它随算例规模稳定上升 (第 6.2 节)。这使「局部」成为一个可以测量的量, 而不是一个人设定的半径。至于哪种结构会产生更大的闭包, 本文没有得到可复现的答案: 在装卸点最小割更窄的布局上, 同型 LU 割边阻断倾向于产生更大的相对闭包, 但两个布局的逐种子取值高度重叠、单个种子即可反向, 五个种子不足以支撑该趋势; 而在同规模的三个布局上, 繁忙走廊阻断下的相对闭包又落在一条很窄的带内 (第 6.7 节)。故本节只作定性陈述, 不作硬贡献。

5 闭包上的有界修复

图 3 总览本文流程: 先以 STRC 测量释放集, 再在闭包上做第 1 级有界协调; 失败则扩域重试。对照臂 R1 仅替换释放集来源, R0+ 则打开热启动搜索——同一无冲突执行器上归因。

5.1 释放、冻结与第 1 级改路

给定释放集 U , 第 1 级协调保持原车辆指派与机器指派, 按原扫描序重放:

- (1) 将扰动禁行窗写入路由层;
- (2) 对 $r \notin U$ 的未来预约预提交到预约表 (外侧冻结); 若外侧已与禁行窗冲突, 则闭包漏网, 直接失败;
- (3) 对释放集内运输: 原车在当前表上重规划空载/满载段并提交;
- (4) 推进机器空闲时刻与工件就绪时刻; 以独立校验器验收, 并禁止修复后占用仍落入禁行窗。

该级不打开种群搜索。同引擎消融 (E3) 固定上述步骤, 仅替换 U 来自 R1 或 R2, 用以把贡献归因于边界定义。

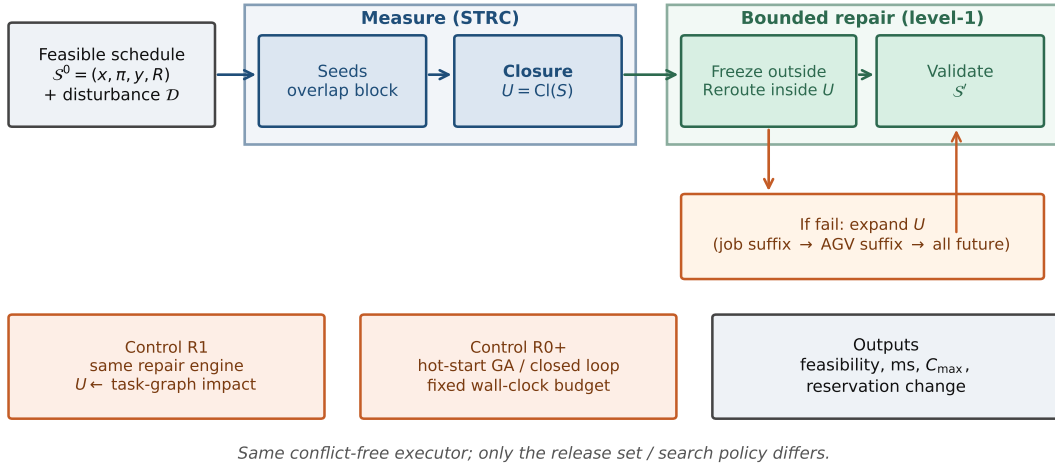


Fig. 3. STRC 有界恢复框架。上排: 扰动下由种子取闭包并改路; 中排: 失败扩域; 下排: R1(任务图边界) 与 R0+(热启动重解) 对照及输出指标。

5.2 失败扩域再修

先声明: 扩域机制本身不是本文首创。Local Rescheduling [8] 已给出「增量式扩张时间窗直至可行」的做法, AOR 的间接影响传播亦是一种扩张 (第 2.1 节)。本文的差别只在扩张沿什么走——沿依赖边而非沿时间窗——这不足以单独作为贡献。本节的作用是把闭包路径补成一个完整可用的流程, 并在第 6.1 节交代它为何必须在边界消融中关闭。

当外侧冻结过紧时, 第 1 级可能因运输-工序衔接等校验失败。此时不立刻求助于全局 GA, 而按下列顺序扩大 U 并重试同一引擎: (i) 并入释放集内各工序的同工件后缀未来预约; (ii) 并入涉及车辆的车后后缀; (iii) 必要时释放全部未来预约。扩域仍是确定性单遍协调。实验表明扩域可将可行率拉至全样本, 耗时仍保持在毫秒至十余毫秒量级 (第 6 节)。需要说明的是, 耗时随扰动比例 φ 并不单调 (表 9): 比例小时初始闭包偏紧、冻结偏多, 扩域更常被触发; 比例大时初始释放已较松、往往一轮成功, 但单轮要重规划的运输更多。两种效应方向相反, 故本文不宣称耗时随 φ 有可预测的走向, 只宣称它不随 φ 崩溃。

5.3 与全局重解的对照

对照臂在相同禁行约束下, 以 S^0 的染色体热启动做固定挂钟预算的闭环搜索 (记 R0+)。它可改派与改序, 因而更可能改善 $C_{\max}$, 但耗时由预算决定。对照取热启动而非冷启动, 是因为对照必须取强者: 冷启动会白白丢掉原解携带的全部信息, 赢它不说明问题。

角色分工是: STRC 路径优先可行性与响应时间, 并在结构上限制释放集; R0+ 优先完工时间。需要预先说明这个「分工」的确切含义——它不是指存在某个预算区间使闭包路径的 $C_{\max}$ 更优。第 6.8 节的实测表明这样的区间不存在: R0+ 在最小预算下已占优, 两条曲线不相交。分工指的是两者被不同的约束选中——当响应必须在毫秒量级、且已下发的时窗承诺不宜撕毁时, 只有闭包路径可用; 当允许秒级停机并接受近乎全盘改写时, 应当用 R0+。「外侧字段逐条不变」只在命题 3 的条件下保证, 实验中作部分观测。

算法 1 与图 3 对应: 闭包界定 U , 第 1 级改路, 失败则按工件/车辆/全部未来依次扩域。

Algorithm 1 基于 STRC 的有界恢复 (第 1 级 + 扩域)**Require:** 排程 S^0 , 扰动 $\mathcal{D}$, 时刻 t_{now} **Ensure:** 排程 S' 或失败1: $S \leftarrow \text{Seeds}(\mathcal{D}); U \leftarrow \text{CI}(S)$ 2: **for** 轮次 = 0, 1, 2, 3 **do**3: $S' \leftarrow \text{REPLAYREROUTE}(S^0, \mathcal{D}, U)$ 4: **if** S' 可行 **then**5: **return** S' 6: **end if**7: 按扩域规则更新 U

▷ 工件后缀 / 同车 / 全部未来

8: **end for**9: **return** 失败**6 实验****6.1 协议**

执行器与算例. 下层路网、时间窗路由与独立校验器固定。扩样矩阵使用**五个算例**: 小例 example_3x3x2、拥堵例 congested_8x4x4, 以及三个 S8x4x4 布局 high、funnel 与 mid。后三者同规模、同柔性, 只差走廊拓扑, 因此它们之间闭包规模的差异可归因于拓扑而非问题尺寸。**种子取十个**, 即 42、7、2024、99、123、13、1、777、31415 与 8, 共 50 个算例-种子对。初始排程由冲突自由启发式解码得到; 同一算例-种子对下各对照臂共享该初始解。

扰动. 除非另注, $t_{\text{now}} = 0.35 C_{\text{max}}(S^0)$ 。E1-E3/E5 在繁忙走廊上施加单窗阻断; E6 在同一 t_{now} 上另加三类扰动; 规模扫描将未来工序按最早未来预约排序, 取前比例 φ 的预约时窗作为微阻断 (不合并)。

对照臂. R1: 任务图影响域 + 第 1 级改路, 全文取 $\theta = 2$ (该值只影响 A 类扰动下 $|U_{R1}|$ 的大小; 由命题 1, B 类上任何有限 θ 都给出空集); R2/STRC: 闭包 + 第 1 级改路 (规模扫描打开失败扩域; E3/E5 关闭扩域); R0+: 热启动闭环搜索, 固定挂钟预算。

一处必须声明的口径. E3 中必须关闭失败扩域。若开着扩域, R1 会靠「一路扩到释放全部未来预约」而偶然成功, 测到的就不再是边界定义的差别, 而是扩域规则的兜底能力。扩域本身有效 (规模扫描中它把可行率拉满), 但它与被测变量正交, 故在边界消融中一律关闭。

6.2 E1: 任务图漏报

表 3 与图 4(左) 汇总命题 1 的扩样结果: 50/50 样本上 $T_{\text{impact}} = 0$ 、种子与闭包非空、阻断后原排程不可行, 结构泄漏均为 0。

按命题 1 之后那段话的提醒, 读这张表时重量应压在后半句上。 $T_{\text{impact}} = 0$ 这一列是定义 2 对 B 类约定的推论, 它把「任务图这一表示覆盖不到走廊事件」显示出来, 但本身不是发现。不平凡的是 feasible_after_block 一列恒为假——排程确实已经不可行。两者合起来才构成反例。

闭包规模随算例规模上升 (16.7 $\rightarrow$ 48.3 $\rightarrow$ 80 余), 但 $\text{CI}/|R|$ 在五个算例上稳定在 0.54–0.58, 未能区分三个 S8x4x4 布局。这与第 6.7 节的结论一致: 本文没有测出一个能可靠预测闭包规模的结构指标, 故不把「闭包规模由布局决定」列为贡献。

6.3 E2: 包含性

结构包含性 (E2a) 在 50/50 样本上成立 (泄漏 = 0), 与命题 2 一致。第 1 级修复 (无扩域) 在 50/50 样本上可行。外侧字段逐条不变 (E2b / 命题 3 的观测形式) 同样在 50/50 样本上成立 (表 4)。

Table 3. E1 漏报 (走廊阻断; 每算例 10 种子, 共 50 对)。|R| 为全表预约数, C1/|R| 为闭包占全表的比例。均值取多种子平均。后三行同规模同柔性, 只差走廊拓扑。

算例	种子数	C1 通过	均 R	均 Seeds	均 C1	C1/ R
example_3x3x2	10	10/10	28.6	5.6	16.7	0.58
congested_8x4x4	10	10/10	85.0	15.0	48.3	0.57
S8x4x4 (high)	10	10/10	150.5	15.5	85.9	0.57
S8x4x4 (funnel)	10	10/10	149.1	15.5	82.1	0.55
S8x4x4 (mid)	10	10/10	147.6	15.0	80.2	0.54
合计	50	50/50	—	—	—	—

Table 4. E2 包含性 (10 种子/算例)。E2a 为结构闭合 (泄漏 = 0), E2b 为闭包外侧预约字段逐条不变。

算例	E2a 闭合	第 1 级可行	E2b 外侧逐条不变
example_3x3x2	10/10	10/10	10/10
congested_8x4x4	10/10	10/10	10/10
S8x4x4 (high)	10/10	10/10	10/10
S8x4x4 (funnel)	10/10	10/10	10/10
S8x4x4 (mid)	10/10	10/10	10/10
合计	50/50	50/50	50/50

需要区分 E2a 与 E2b 说的不是一回事, 满通过并不使命题 3 升格为充要。E2a 是图上的性质, 由命题 2 保证, 满通过是应当的; 若不满通过, 说明实现与定义不符。E2b 是修复之后的性质, 它检验的是命题的四条前提在本文协议下是否成立——而不是这些前提可以省略。特别地, 前提 (ii) 要求外侧预约逐条预提交成功; 若某条外侧预约与禁行窗重叠, 预提交即失败, 那说明种子集不完备 (注 3), 此时命题不适用。本文的实验落在前提成立的一侧, 不等于任何设定下都会落在那一侧。

注 4 (一处曾经把实现缺陷读成边集缺陷的教训)。这一栏在早前的批次上只有 24/50。当时的解读是「依赖边集未能覆盖若干弱耦合」, 并据此把加细边集列为首要后续工作。复核后该解读是错的。释放集是一个预约集合, 而重放时是否改路却按工序判定——同一工序的空载段与满载段只要有一段落在闭包内, 整道运输就被重规划。于是当只有满载段进入闭包时, 同工序的空载段也被改写, 而它的任务标签并不在释放集内, 遂被记为外侧漂移。26 个不通过格上的 84 条漂移预约无一例外都是这种同工序空载段, 其中 71 条 (85%) 在 t_{now} 之前就已执行完毕——改写它同时违反假设 A2。改为按段判定 (已执行完或未被释放的空载段沿用原计划, 只重规划真正落在释放集内的段) 之后, 漂移归零。换言之, 此前测到的不是边集对物理耦合的覆盖度, 而是命题 3 前提 (iii) 「仅对 U 内运输调用路由器」在实现上没有被真正满足。记下这一条, 是因为它示范了一种容易发生的误判: 把实现与定义的粒度错配, 读成模型对世界的刻画不足。

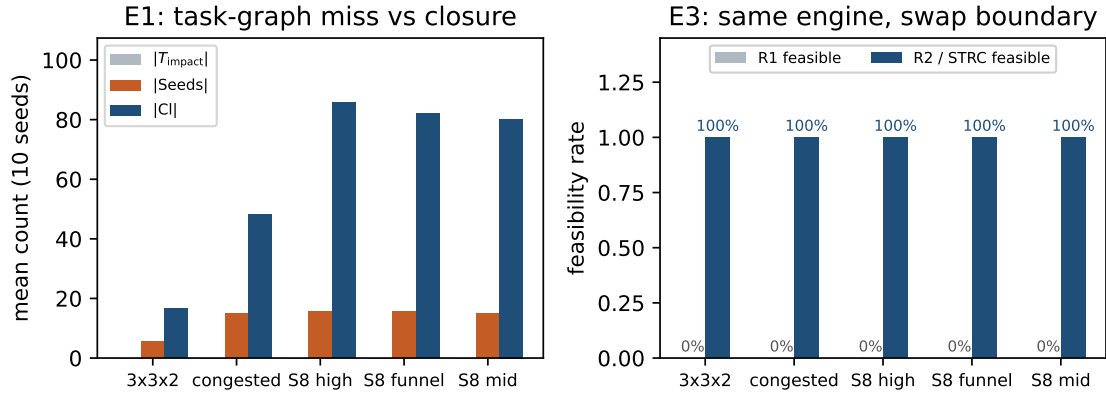
6.4 E3: 边界消融

表 5 与图 4(右) 固定第 1 级改路并关闭失败扩域。B 类上 50/50 样本 R1 释放集为空且不可行, R2 全部可行。

这是本文最强的一组对照, 但它的说服力来自干净而不是来自数字大: 同一个修复引擎、同一份初始解、同一个 t_{now} 、同一批种子、同一段阻断窗, 唯一的差别是释放集合怎么算。因此 0/50 对 50/50 这个反差不能归因于改路启发式的优劣——R1 侧根本没有触发任何改路, 它的释放集是空的, 第 1 级改路对空集是恒等操作, 于是原排程原封不动地留在被阻断的走廊上。换句话说, R1 的失败方式不是「修得不好」, 而是「判断为无需修」。修复耗时中位 2.8 ms、最大 5.2 ms。

Table 5. E3 边界消融 (无扩域, 5 算例 $\times$ 10 种子)。miss_B 指该格上 R1 释放集为空而 R2 非空。

算例	miss_B	R1 可行	R2 可行	均 $ U_{R2} $	R2 耗时/ms
example_3x3x2	10/10	0/10	10/10	16.7	0.73
congested_8x4x4	10/10	0/10	10/10	48.3	1.96
S8x4x4 (high)	10/10	0/10	10/10	85.9	3.77
S8x4x4 (funnel)	10/10	0/10	10/10	82.1	3.05
S8x4x4 (mid)	10/10	0/10	10/10	80.2	3.27
合计	50/50	0/50	50/50	—	—

Fig. 4. E1/E3 扩样可视化 (5 算例 $\times$ 10 种子均值)。左: 任务图影响域为空而种子与闭包非空; 右: 同改路引擎下仅换边界时 R1 可行率为 0、R2 为 100%。Table 6. E6 扰动类型 $\times$ 边界定义 (50 对/类, 共 200 次测量)。 $|R^*|$ 为 t_{now} 之后的活预约数。「R1 为空」即任务图边界判该扰动无需重调度的格数。

扰动	类	R1 为空	均 $ U_{R1} $	均 $ CI $	$CI/ R^* $
走廊阻断	B	50/50	0.0	62.6	0.90
路段降速	B	50/50	0.0	62.6	0.90
车辆故障	A	0/50	31.1	53.0	0.75
机械臂故障	A	0/50	42.8	62.7	0.90

6.5 E6: 扰动类型与边界定义交叉

E1–E3 只用了走廊阻断一种扰动。本小节把扰动类型这一维展开, 在同一 50 对上另加路段降速、车辆故障与机械臂故障三类, 填满表 6 这张覆盖矩阵。

先说这一组只测什么、不测什么。四类扰动中只有走廊阻断被修复引擎正确建模——阻断以强制占用注入路由层 (第 5.1 节)。降速在当前实现中被当作整段阻断处理, 是一个过近似; 车辆故障与机械臂故障则没有对应的注入通道, 重放时那台车/那台机器仍然可用。若对后三类照跑「可行率」, 得到的会是假阳性。因此 E6 只报边界: 任务图影响域有多大、预约闭包有多大、后者是否包含前者。这恰好是覆盖矩阵要回答的问题——「任务图边界看不看得见这类扰动」——而它不需要修复语义。

Table 7. E4 闭包占比 $CI/|R|$ (LU 割边阻断; 种子 42, 7, 2024, 99, 123)。

布局	42	7	2024	99	123	中位
funnel	0.254	0.181	0.296	0.309	0.180	0.254
high	0.156	0.204	0.372	0.229	0.184	0.204

三点读数。第一, **A/B 二分**在 200 次测量上无一例外: 两类 B 扰动的任务图边界全部为空, 两类 A 扰动全部非空。第二, 在全部 100 次 A 类测量上 $U_{R1} \subseteq CI$ ——这说明改用闭包边界在 A 类上不会丢失任务图能看见的东西。包含在 81/100 上是严格的, 其余 19 格两者恰好相等 (此时闭包沿依赖边没有走出任务图影响域诱导的那个集合), 不是包含失败。代价是释放集平均变大: 车辆故障 $31.1 \rightarrow 53.0$, 机械臂故障 $42.8 \rightarrow 62.7$ 。结合第一点, 闭包边界在这四类扰动上是任务图边界的一致加强, 而非另一种取舍。第三, **降速与阻断两行逐格相同**, 在全部 50 对上闭包规模一致。这不是两次独立的验证: 本文的种子规则对二者都是「与失效时窗重叠的活预约」, 故它们按构造给出同一个种子集, 进而给出同一个闭包。该行证实的是任务图边界对降速同样为空, 不是对 B 类结论的重复确认。

这张表也给出了不作最小性主张的实测依据 (注 2)。走廊阻断下闭包覆盖 t_{now} 之后活预约的约 90%; 相对「释放全部未来」这一平凡上界只省下一成。有界修复真正省下的是不动过去与不重开搜索, 而不是把未来切得很小。第 6.2 节按全表统计的 $CI/|R| \approx 0.57$ 与此不矛盾: 全表包含 t_{now} 之前已经执行完毕、按假设 A1 本就不可改的那一半。两个口径都给出, 以免读者只看到有利的那个。

6.6 案例甘特

E1–E3 给出的是跨种子的计数与可行率。为把「任务图覆盖不到、闭包可修」落到时间轴上, 图 5 取 example_3x3x2、种子 42 的一例走廊阻断 (与 E2/E3 同协议: $t_{\text{now}} = 0.35 C_{\text{max}}$, 繁忙走廊单窗阻断, 第 1 级改路、无扩域)。

该例上 $T_{\text{impact}} = \emptyset$, 而 $|Seeds| = 6$ 、 $|CI| = 16$ 。图 5(a) 为扰动前原排程的机器甘特: 蓝色工序的运输预约落入闭包 (将被释放改路), 灰色工序在闭包外 (预提交冻结); 红色虚线为 t_{now} , 浅红带为阻断时窗。图 5(b) 为修复后: 闭包内工序时间轴重排, 排程在阻断下恢复可行, $C_{\text{max}} : 57 \rightarrow 86$, 耗时亚毫秒级。本图不用于宣称完工时间最优——它只说明统计结论对应何种局部改动形态; 完工时间与全局重解的权衡见随后的 E5 与规模扫描。

6.7 E4: 结构 (探索)

在 LU 割边阻断下, funnel 与 high 的闭包占比随种子波动 (表 7)。方向上确实是 funnel 更高——中位 0.254 对 0.204, 均值 0.244 对 0.229——但两组取值高度重叠: funnel 落在 $[0.180, 0.309]$, high 落在 $[0.156, 0.372]$, 后者的区间把前者整个包住, 且单个种子即可反向 (2024 一列上 high 的 0.372 反高于 funnel 的 0.296)。五个种子、两个布局, 这个差距不足以支撑任何结论。第 6.2 节按繁忙走廊阻断协议在三个同规模布局上得到的 $CI/|R|$ 亦落在 0.54–0.57 的窄带内, 同样未能区分布局。

因此本文不把「闭包规模由争用结构决定」列为贡献, 只作一句观察: 闭包规模随算例规模稳定上升, 但本文尝试过的两个结构指标 (最小割、漏斗占比) 都没有在布局之间做出可复现的区分。这一失败本身是有信息的——它说明「哪种布局更容易产生大闭包」需要比最小割更细的刻画, 列为后续工作。

6.8 E5: 与全局重解的权衡

本小节的设计意图原是找一个交叉点: 预算紧时有界修复占优, 预算放宽后热启动全局重解反超。该交叉点不存在, 本文如实报告这一结果, 并据此改写机制层的定位。

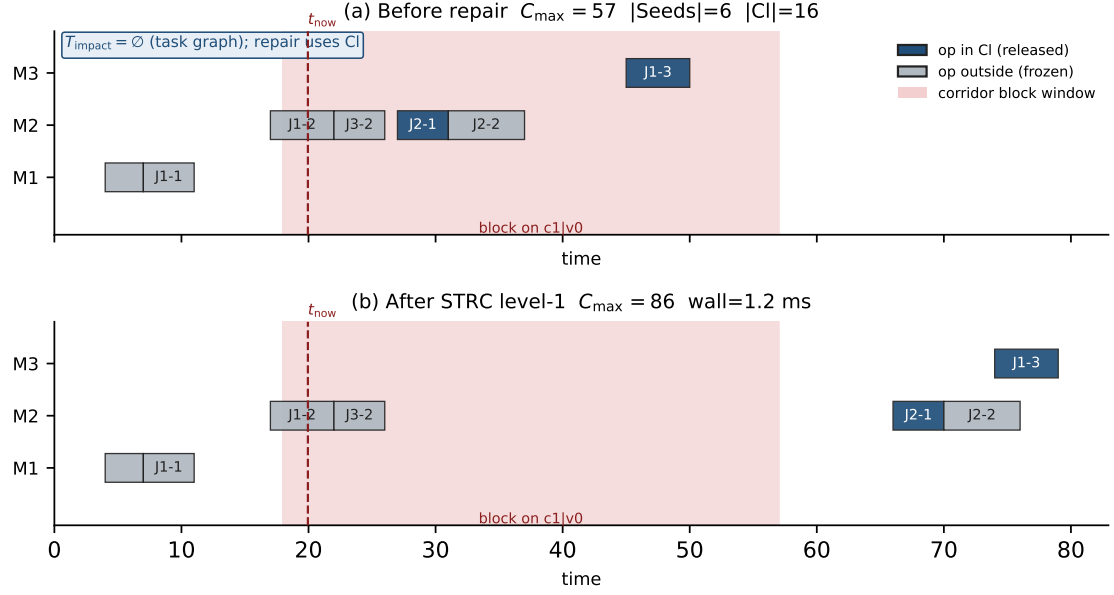


Fig. 5. 案例甘特 (example_3x3x2, seed 42)。 (a) 阻断前原排程, 按是否属于 CI 着色; (b) STRC 第 1 级修复后。任务图影响域为空; 蓝色工序对应闭包内运输被改路。

Table 8. E5 与热启动全局重解的权衡 (3 种子均值)。「改动比例」为被改写的预约占全表之比。 $C_{\max}$ 一列 R0+ 全区间更优。

算例	预算/s	R0+ $C_{\max}$	R2 $C_{\max}$	改动比例 R0+ / R2
congested_8x4x4	0.2	212.7	299.0	1.00 / 0.59
	1.0	160.7	299.0	1.00 / 0.59
	2.0	140.0	299.0	1.00 / 0.59
example_3x3x2	0.2	45.7	88.7	0.97 / 0.66
	1.0	45.3	88.7	0.97 / 0.66
	2.0	45.3	88.7	0.97 / 0.66

表 8 与图 6 在两个算例上对种子 {42, 7, 2024} 扫描 R0+ 预算 {0.2, 1, 2} s, 共 $2 \times 3 \times 3 = 18$ 个预算点。R0+ 即使在最小预算 0.2 s 下也已优于 R2, 且随预算继续拉开; 在全部 18 个点上无一例外。

据此改写角色分工的表述。有界修复在时间 (低两至三个数量级) 与 稳定性 (改动 59%–66% 的预约, 而 R0+ 改动 97%–100%) 两个维度上大幅占优, 在解质量上全区间劣势。诚实的定位是:

有界修复不是「更好的重调度」, 而是「必须立刻给出可执行方案、且不愿撕毁已下发承诺时」的那一档。若允许秒级停机并接受近乎全盘改写, 热启动全局重解在完工时间上占优。

需要说明为什么不换指标绕过去。若改用「相对初始解的 $C_{\max}$ 退化率」或「 $C_{\max}$ 与改动比例的加权和」, R2 都会好看一些, 但那是在事后挑一个对自己有利的标量化方式。 $C_{\max}$ 是本领域的主指标, 本文在它上面输了, 就按它报。这条读数也约束了前文的用词: 第 5.3 节所说的「互补」, 指的是两者在响应时间这一约束下各自占据的区间, 不是指存在某个预算区间使 R2 更优。

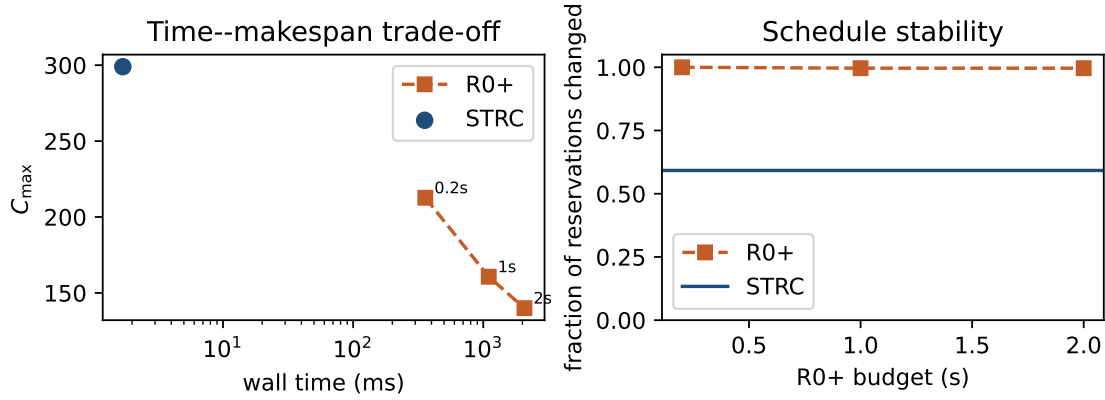


Fig. 6. E5:R0+ 预算扫描与 STRC 定点 (congested_8x4x4, 3 种子均值)。左: 耗时- $C_{\max}$, 两条曲线不相交; 右: 预约改动比例。

Table 9. 扰动规模对照 (congested_8x4x4, 10 种子平均)。STRC 含扩域;R0+ 预算 2 s。

φ	STRC 可行	STRC ms	R0+ $C_{\max}$	耗时比
0.10	100%	7.9	99.4	393×
0.25	100%	9.7	104.0	377×
0.50	100%	6.7	110.2	422×
0.75	100%	7.9	116.9	287×
1.00	100%	9.6	123.5	223×

6.9 扰动规模扫描

表 9 与图 7 报告 φ 扫描 (congested_8x4x4, 10 种子; R0+ 预算 2 s; STRC 含失败扩域)。两臂可行率均为 100%; STRC 耗时约 7–10 ms, R0+ 约 2 s; $C_{\max}$ 仍全由 R0+ 更优, 与第 6.8 节一致。小例 example_3x3x2 上同样每格可行 (10 种子 $\times$ 五个 φ 值), STRC 约 1 ms 量级, 加速比 110 $\times$ –4341 $\times$ (跨两个算例的逐格极值)。

这里 STRC 打开了失败扩域, 故与 E3/E5 不可直接比较: 扩域是把可行率免到 100% 的机制, 而它本身并非本文首创 (第 2.1 节)。本表的用途是说明, 即使在扰动比例升至 $\varphi = 1$ (全部未来工序受扰) 时, 闭包路径仍停留在十余毫秒量级, 响应时间不随扰动规模崩溃。

6.10 小结

- 问题层 (E1) 与边界消融 (E3) 在 50 个算例-种子对上无一例外成立; 案例甘特把同一协议落到工序时间轴。
- 覆盖矩阵 (E6) 在四类扰动 $\times$ 50 对上给出完整读数: A/B 二分无例外, 且 A 类上闭包边界一致包含任务图边界。
- 结构包含性 (E2a) 与外侧字段逐条不变 (E2b) 均满通过; 但这只说明命题 3 的四条前提在本文协议下成立, 命题仍只作充分条件 (注 4 记录了前提 (iii) 曾如何在实现上被悄悄破坏)。
- 与全局重解的关系是在响应时间约束下的分工, 不是质量上的互有胜负: 毫秒 vs 秒、改动 59%–66% vs 97%–100%, 但 $C_{\max}$ 全区间劣势、无交叉。
- 闭包覆盖约 90% 的活预约, 本文不作最小性主张。
- 没有测出能区分布局的结构指标 (E4), 故闭包规模的可预测性不作贡献。

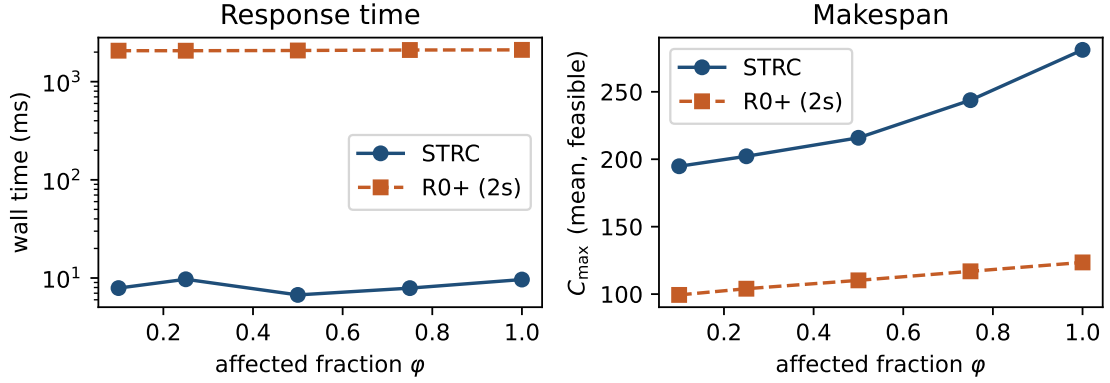


Fig. 7. 扰动规模扫描 (congested_8x4x4, 10 种子): 响应时间 (对数轴) 与完工时间随 ϕ 变化。

7 结论

7.1 结论

本文指认了一类任务图覆盖不到的级联损坏, 并将影响边界重新定义在时空预约闭包 (STRC) 上。在 5 算例 $\times$ 10 种子共 50 对上: 走廊阻断的任务图影响域 50/50 为空而原排程确已不可行; 同一修复引擎下仅替换边界定义时, 任务图边界 0/50 恢复可行而闭包 50/50; 闭包的结构闭合性 50/50 成立, 闭包外侧预约字段的逐条不变 50/50 成立; 四类扰动 $\times$ 两种边界的覆盖矩阵上 A/B 二分无例外, 且 A 类上闭包边界一致包含任务图边界。

主张的边界也已划清。让行关系这一建模对象、「局部重规划 + 外侧冻结」这一机制形态、以及「失败后扩大释放域」这一策略, 分别在铁路 alternative graph、多智能体路径规划与 Local Rescheduling 中早已存在, 本文不作创新声称 (第 2.5 节)。本文主张的是三件事: 该边界在两图分立的模型层级上被划错了; 改画之后它是一个由图结构唯一确定、有闭合性、可测量的对象而非启发式半径; 以及这个对象是一次测量而不是一个决策。

7.2 局限

- (1) 完工时间上全区间劣于热启动全局重解, 且不存在交叉点。在 18 个预算点上无一例外。本文买到的是响应时间与稳定性, 卖掉的是解质量; 定位已按此改写 (第 6.8 节), 但这仍是一条实质限制。
- (2) 命题 3 仍只是充分条件, 50/50 不等于充要。实验说明的是四条前提在本文协议下成立, 不是它们可以省略。前提 (iii) 尤其脆弱: 只要实现以比预约更粗的粒度决定重规划范围, U 外的占用就会被顺带改写 (注 4)。本文没有给出一个能自动检出该类粒度错配的手段, 这一条只能靠逐条比对外侧字段来事后发现。
- (3) 依赖边集是否穷尽物理耦合, 本文并未验证。E2b 满通过检验的是修复实现与释放集定义是否一致, 而不是 G_R 的四类边是否覆盖了世界上一切可能的传播通道——后者需要一个独立于 G_R 的耦合真值, 本文没有。注 2 的过近似讨论只说明边集可能偏多, 反方向仍未检验。
- (4) 不作最小性主张, 且闭包确实不小。闭包覆盖 t_{now} 之后活预约的约 90% (注 2)。相对「释放全部未来」这一平凡上界, 结构化释放省下的不足一成。
- (5) 只有走廊阻断被修复引擎正确建模。E6 的另外三类扰动只报边界、不报可行率: 降速被当作整段阻断 (过近似), 两类故障没有注入通道 (第 6.5 节)。因此覆盖矩阵在「边界」一维是满的, 在「修复」一维仍只有一格。

- (6) 恢复动作受限于假设 A5。保持原机器指派与扫描序, 只允许改路与扩域。换车、改派、改序属于升级阶梯的后续级, 本文未评估; 放开它们即向全局重解退化, 「有界」也随之失去边界。
- (7) 单次扰动, 信息完备。由假设 A2、A3, 扰动在 t_{now} 完全已知且不叠加。扰动流、预测误差与滚动响应均未涉及。
- (8) 闭包规模的结构可预测性未能建立。最小割与漏斗占比两个指标都没有在同规模布局之间做出可复现的区分 (第 6.7 节)。
- (9) 与静态一体化工作共享代码栈。下层路网、时间窗路由与校验器复用同一实现, 这保证了归因的干净, 但也意味着结论尚未在第二套独立实现上复现。

7.3 后续工作

按上述局限排序, 最值得做的四件事如下。为依赖边集建立一个独立的检验手段: E2b 现在只能确认实现与定义自治, 无法确认 G_R 的四类边是否覆盖了全部物理传播通道。可行的做法是构造扰动-响应的差分实验——只改动闭包内某一条预约, 观察闭包外是否出现本不该出现的连锁——从而在不预设耦合真值的前提下探测漏边。向最小释放集逼近: 在固定修复能力下定义最小性, 并给出闭包与之的差距——这是把「有界」从充分条件推进到紧界的必经一步; 当前闭包覆盖约 90% 的活预约, 差距很可能不小。为降速与故障补上注入通道, 使覆盖矩阵在修复一维也填满; 降速需要路由层支持分段 τ , 车辆故障需要把该车从可用车队中摘除并允许换车, 后者同时会触及假设 A5。扰动流与滚动响应: 放开 A3, 考察闭包在连续扰动下是否会累积膨胀, 以及两次修复之间是否需要显式的「归位」步骤。

References

- [1] Riyad J. Abumaizar and Joseph A. Svestka. 1997. Rescheduling job shops under random disruptions. *International Journal of Production Research* 35, 7 (1997), 2065–2082. doi:10.1080/002075497195074
- [2] M. Selim Aktürk and Elif Görgülü. 1999. Match-up scheduling under a machine breakdown. *European Journal of Operational Research* 112, 1 (1999), 81–97. doi:10.1016/S0377-2217(97)00396-2
- [3] James C. Bean, John R. Birge, John Mittenenthal, and Charles E. Noon. 1991. Matchup scheduling with multiple resources, release dates and disruptions. *Operations Research* 39, 3 (1991), 470–483. doi:10.1287/opre.39.3.470
- [4] Manuel Dalcastagné, Andrea Mariello, and Roberto Battiti. 2020. Reactive Sample Size for Heuristic Search in Simulation-based Optimization. *arXiv preprint arXiv:2005.12141* (2020).
- [5] Andrea D'Ariano, Dario Pacciarelli, and Marco Pranzo. 2007. A branch and bound algorithm for scheduling trains in a railway network. *European Journal of Operational Research* 183, 2 (2007), 643–657. doi:10.1016/j.ejor.2006.10.034
- [6] Jie Fu, Bin Yang, Zhixing Chang, Yuanrong Zhang, Jiarui Wang, Xiaotong Wang, and Lei Wang. 2025. Integrated Production-Logistics Scheduling in Flexible Assembly Shops Using an Improved Genetic Algorithm. *Machines* 13, 12 (2025), 1090.
- [7] Jeonghyeon Kim and Junwoo Kim. 2026. Congestion-Aware Scheduling for Large Fleets of AGVs Using Discrete Event Simulation. *Electronics* 15, 1 (2026), 139. doi:10.3390/electronics15010139
- [8] Jürgen Kuster, Dietmar Jannach, and Gerhard Friedrich. 2007. Local Rescheduling: A novel approach for efficient response to schedule disruptions. In *2007 IEEE Symposium on Computational Intelligence in Scheduling (CISched)*. IEEE, 79–86. doi:10.1109/SCIS.2007.367673
- [9] Ruiqi Li, Jianlin Mao, Xing Wu, Wenna Zhou, Chengze Qian, and Haoshuang Du. 2026. A New Framework for Job Shop Integrated Scheduling and Vehicle Path Planning Problem. *Sensors* 26, 2 (2026), 543. doi:10.3390/s26020543
- [10] Rong-Kwei Li, Yu-Tang Shyu, and Sadashiv Adiga. 1993. A heuristic rescheduling algorithm for computer-based production scheduling systems. *International Journal of Production Research* 31, 8 (1993), 1815–1826. doi:10.1080/00207549308956824
- [11] Nir Malka, Guy Shani, and Roni Stern. 2024. Online Planning for Multi Agent Path Finding in Inaccurate Maps. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 10214–10221. doi:10.1109/IROS58592.2024.10801975
- [12] Alessandro Mascis and Dario Pacciarelli. 2002. Job-shop scheduling with blocking and no-wait constraints. *European Journal of Operational Research* 143, 3 (2002), 498–517. doi:10.1016/S0377-2217(01)00338-1
- [13] Leilei Meng, Weiyao Cheng, Chaoyong Zhang, Kaizhou Gao, Biao Zhang, and Yaping Ren. 2025. Novel CP Models and CP-Assisted Meta-Heuristic Algorithm for Flexible Job Shop Scheduling Benchmark Problem with Multi-AGV. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 55, 11 (2025), 8455–8468. doi:10.1109/TSMC.2025.3604355

- [14] Djamila Ouelhadj and Sanja Petrovic. 2009. A Survey of Dynamic Scheduling in Manufacturing Systems. *Journal of Scheduling* 12, 4 (2009), 417–431. doi:10.1007/s10951-008-0090-8
- [15] Haoxiang Qin, Yi Xiang, Yuyan Han, Yuting Wang, Junqing Li, and Quanke Pan. 2026. A Knowledge Region Selection Enhanced Quality-Diversity Algorithm for Real-World Flexible Job Shop Scheduling with Automated Guided Vehicles Transportation. *Engineering Applications of Artificial Intelligence* 164 (2026), 113352. doi:10.1016/j.engappai.2025.113352
- [16] Velusamy Subramaniam and Amritpal Singh Raheja. 2003. mAOR: A heuristic-based reactive repair mechanism for job shop schedules. *The International Journal of Advanced Manufacturing Technology* 22, 9-10 (2003), 669–680. doi:10.1007/s00170-003-1601-6
- [17] Jiachen Sun, Zifeng Xu, Zhenhao Yan, Lilan Liu, and Yixiang Zhang. 2023. An Approach to Integrated Scheduling of Flexible Job-Shop Considering Conflict-Free Routing Problems. *Sensors* 23, 9 (2023), 4526. doi:10.3390/s23094526
- [18] Jiri Švancara, Marek Vlk, Roni Stern, Dor Atzmon, and Roman Barták. 2019. Online Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 7732–7739. doi:10.1609/aaai.v33i01.33017732
- [19] Roel W. M. van Os and Twan Basten. 2026. An Exact Decomposition-Based Approach to the Conflict-Free-Transportation-Constrained Flexible Job-Shop Scheduling Problem. *Computers & Operations Research* 187 (2026), 107342. doi:10.1016/j.cor.2025.107342
- [20] Guilherme E Vieira, Jeffrey W Herrmann, and Edward Lin. 2003. Rescheduling Manufacturing Systems: A Framework of Strategies, Policies, and Methods. *Journal of Scheduling* 6, 1 (2003), 39–62. doi:10.1023/A:1022235519958
- [21] Bin Xin, Sai Lu, Qing Wang, and Fang Deng. 2025. A Review of Flexible Job Shop Scheduling Problems Considering Transportation Vehicles. *Frontiers of Information Technology & Electronic Engineering* 26, 3 (2025), 332–353. doi:10.1631/FITEE.2300795
- [22] Yannik Zeiträg and José Rui Figueira. 2025. A novel machine-learning rolling horizon heuristic for dynamic lot-sizing and job shop scheduling problems. *International Journal of Production Research* 63, 12 (2025), 4563–4589.
- [23] Yulun Zhang, He Jiang, Varun Bhatt, Stefanos Nikolaidis, and Jiaoyang Li. 2024. Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*.
- [24] Ziyang Zhao, Shixin Liu, MengChu Zhou, Xiwang Guo, and Liang Qi. 2019. Decomposition method for new single-machine scheduling problems from steel production systems. *IEEE Transactions on Automation Science and Engineering* 17, 3 (2019), 1376–1387.